

Gestão de Consultas Médicas e Pessoas

Olá a todos!

O nosso objetivo é criar uma aplicação que permita a gestão de consultas médicas, envolvendo médicos e pacientes. O sistema será composto por dois microsserviços, um para a gestão de pessoas e outro para a gestão de consultas. O frontend deverá ser capaz de comunicar com estes serviços para realizar todas as operações necessárias.

A vossa missão é desenvolver o backend de ambos os serviços e o frontend de forma a cumprir os requisitos de negócio.

Gestão de Pessoas (Médicos e Pacientes)

1. Registo e Atualização de Pessoas:

- Um médico pode ter uma ou mais especialidades associadas.
- As especialidades médicas, assim como os géneros e os tipos de documento de identificação (Cartão de Cidadão ou Passaporte), devem ser dados predefinidos no backend. O frontend deverá obter estas listas do backend para apresentá-las ao utilizador como possíveis opções.
- Ao criar um novo médico ou paciente, o sistema deverá gerar um identificador único de forma automática. O utilizador não deve ter de fornecer este valor. No entanto, este ID será disponibilizado.
- As pessoas devem ter a sua data de nascimento registada. Ao criar ou atualizar a pessoa, o utilizador indicará a data de nascimento, mas o sistema apenas deverá apresentar a idade atual.
- Todas as pessoas devem ter a possibilidade de ter ou não um número de telemóvel.
- O sistema deve garantir que não existem duas pessoas diferentes com o mesmo tipo de documento e o mesmo número de identificação. Por exemplo, não podem existir dois pacientes com Cartão de Cidadão e o número 1234567.

2. Auditoria:

- Queremos ter um registo automático e visível apenas para a equipa técnica (não para o utilizador final) que indique a data e a hora em que um determinado registo (médico, paciente, consulta) foi criado e a última vez que foi atualizado.

Gestão de Consultas

1. Agendamento de Consultas:

- **Fase 1:**
 1. Ter um docker file
 2. Ter um docker-compose
- **Fase 2:**
 1. Endpoint para criar um Appointment seguindo o Data Type para obter o mesmo
 2. Endpoint para obter todos os Appointments
 3. Endpoint para obter apenas um Appointment
 4. Não teremos uma BD
 5. Temos de seguir a lógica de Controller->Service->Repository
 6. Porém os dados serão guardados numa Data Structure no Repository Layer
- **Fase 3:**
 1. Docker-compose com BD
 2. Utilizarem a BD para lógica atual
- **Fase 4:**
 1. Integrar com os serviços Java (Pasta Integrations)
 1. Ao inserir quero garantir que o Doctor e Patient Ids existem
 2. Ao obter uma Consulta quero devolver os seguintes dados do Serviço Java:
 1. PatientName, Patient Insurance number, DoctorSpecialties, Doctor Name
- **Fase 5:**
 1. O paciente deverá ter a possibilidade de pesquisar médicos com base na sua especialidade.
 2. Quando o paciente realiza esta pesquisa, o sistema deverá retornar apenas as informações essenciais sobre o médico: o seu ID, nome e especialidades.

Tecnologias e Arquitetura

- **Microserviços:** O projeto deverá ser dividido em dois serviços:
 - **Person Management:** Responsável pela gestão de médicos e pacientes, a ser implementado em **Java com Spring**.
- **Appointment Management:** Responsável pela gestão das consultas, a ser implementado em **.NET**.
- **Comunicação entre serviços:** A comunicação do microserviço de Gestão de Consultas com o de Gestão de Pessoas deverá ser feita através de **REST**.
- **Base de Dados:** Os microserviços irão utilizar a base de dados em memória **H2** para persistência de dados.
- **Implementação Robusta:** A implementação deve incluir **tratamento de exceções** adequado para lidar com erros de forma controlada e um sistema de **logging** detalhado para facilitar a investigação e correção de problemas, além da monitorização da aplicação.

- **Testes de API:** Para testar os endpoints dos microsserviços antes de ter os frontends implementados, sugerimos a utilização de ferramentas como **Swagger**, **Bruno**, ou **Insomnia**.
- **Frontends:** Deverão ser desenvolvidos frontends, um em **React** e outro em **Angular**, que consumam as APIs de backend para realizar as funcionalidades descritas.

O nosso objetivo é ter uma aplicação funcional e bem estruturada. O primeiro passo será a criação de um **diagrama de classes** que represente o modelo de dados dos vossos serviços.

Para isso, sugerimos a utilização de ferramentas baseadas em texto como **PlantUML** ou **Mermaid**, pois permitem manter o diagrama como código.

É fundamental que se concentrem na **estrutura e nas relações** entre as classes. **Não se preocupem em definir os tipos de dados** nesta fase inicial.

Este diagrama servirá como o mapa para a implementação.

Boa sorte!